

Architectural Primer for CeTTa, MeTTa, PeTTa, and RhoCalc

A conceptual synthesis for future design work

2026-05-15

Abstract

This note consolidates a set of architectural principles that emerged from design and review work around CeTTa, MeTTa, PeTTa, and RhoCalc. It is not a status report, product plan, or historical record. It is a conceptual primer intended to preserve the main ideas needed for future work: how to separate core semantics from profiles and extensions; how to integrate proof-aware optimization with practical runtime engineering; how to design concurrency and process-calculus support without poisoning the host language; how to structure testing, benchmarking, and productization; and how to choose boundaries that remain stable under later implementation pressure. The intended use is simple: a new engineering or research thread should be able to read this document and recover the main architectural frame without re-deriving it from scattered discussions.

Contents

1. Purpose and scope

The main problem is not merely to build one language implementation. The problem is to build a family of semantically related systems that can evolve without collapsing into ambiguity, ad hoc optimization, or accidental dialects. The systems in scope are:

- **CeTTa**: a systems-oriented runtime and implementation substrate.
- **MeTTa**: a symbolic language family with rewriting, nondeterminism, spaces, and reflection.
- **PeTTa**: a Prolog-oriented implementation path that exploits logic-programming strength while carrying MeTTa-like semantics.
- **RhoCalc**: a pure process-calculus layer for concurrency, communication, and name generation.

The design target is not “one true implementation.” The design target is a stack with:

- a small and explicit semantic core,
- named profiles for genuine semantic variation,
- adapters between host surfaces and core semantics,
- runtime substrates that can be optimized without silently changing meaning,
- and enough discipline around tests, benchmarks, and releases that the system can eventually become dependable outside a research lab.

2. The master separation: semantics, syntax, profile, runtime, runner

A large fraction of later confusion can be prevented by keeping five concepts separate.

2.1. Semantics

Semantics is the mathematical or operational contract of a language fragment. It answers the question: what does this term mean?

2.2. Syntax

Syntax is only the concrete encoding of terms. A single semantic core may have multiple syntaxes: for example a MeTTa-like S-expression syntax and a Rholang-style concrete syntax.

2.3. Profile

A profile is an explicit named choice about supported semantics, optional features, or compatibility behavior. Profiles are necessary when the ecosystem is still exploring, but they must be explicit and machine-readable.

2.4. Runtime substrate

The runtime substrate is the execution machinery: term representation, space indices, query engines, channels, worker pools, mutation logs, table managers, and so on.

2.5. Runner policy

A runner is an explicit execution policy on top of a relation or evaluator: repeated stepping, deterministic scheduling, throughput mode, bounded demand, etc. A runner must not silently redefine the language.

2.6. The guiding rule

A stable architecture should be expressible as:

Semantics first, syntax second, profile third, runtime fourth, runner fifth.

Violating this ordering creates the usual pathologies: host interfaces define backend meaning; syntactic convenience bleeds into semantics; optimization shortcuts become undocumented language features; or a runner masquerades as the core language.

3. Standardization without freezing too early

The most defensible process for an experimental language family is not full unification now and not laissez-faire forever. The right pattern is:

1. prototype a small core,
2. write a living spec for that core,
3. keep disputed behavior in explicit profiles or extensions,

4. require tests and at least one independent semantic check before promoting behavior into the core,
5. periodically publish stable snapshots.

The most important practical rule is:

Same syntax must not silently mean different semantics. If semantics vary, the selected profile must be explicit.

This suggests a clean ladder:

- **Experimental:** one implementation may ship it behind a flag; no portability claim.
- **Draft profile:** prose semantics and tests exist; portability still provisional.
- **Candidate core:** a test suite and either a second implementation or an independent validator exist.
- **Ratified snapshot:** included in a stable release or dated semantic snapshot.

For MeTTa-family work, the main risk is not the lack of a standard. The main risk is letting implementation identity decide semantics while all sides still claim to implement the same language.

4. The layered architecture for MeTTa-family systems

4.1. Core language versus host language

The right architecture is not to define every sublanguage in terms of every other one. Instead:

- each core should have its own semantic object,
- host languages should expose thin adapters onto that object,
- and projections between layers should be explicit.

This matters especially for concurrency. MeTTa can interleave with process-calculus concepts, but that does not mean the Rho core should be defined in terms of MeTTa nondeterminism, nor that MeTTa should inherit process-calculus runner policy by accident.

4.2. CeTTa as substrate

CeTTa should be treated as the shared implementation substrate, not as the owner of every semantic object. In practice this means:

- shared memory management, canonical term IDs, indexing, logging, metrics, and build/test infrastructure,
- but separate semantic kernels where needed.

4.3. Profiles rather than silent dialects

For MeTTa-family work, the clean split is:

- **core:** the small normative surface,
- **compat:** useful but non-core compatibility behaviors,
- **extension:** explicit extra surfaces and policies,

- **workload/support:** examples, probes, benchmark harnesses, and diagnostics, which should not be mistaken for core semantics.

5. HE compatibility over PeTTa: the right conceptual model

The right mental model for HE-on-PeTTa is not “pretend everything is HE.” It is:

preserve a strict HE-compatible core, keep compatibility layers explicit, and let PeTTa-specific power remain visible as extension rather than secretly redefining the core.

5.1. Translator-generated HE witnesses

A strong move is to generate HE-spec witness versions of example workloads. This prevents “PeTTa-he still has many PeTTa powers” from hiding semantic drift. The translator becomes part of the semantics discipline, not merely a convenience tool.

5.2. Three execution fragments matter

The remaining performance and architecture work in HE-on-PeTTa falls naturally into three fragments.

1. **Deterministic compiled fragment:** pure, deterministic, no observable multivalued behavior, no mid-call mutation of the callable/type world. This should lower to a direct compiled Prolog ABI with guarded deoptimization fallback.
2. **Grouped mutation fragment:** repeated `&self` mutation where intermediate states are not observed. This wants batched write-sets and commit barriers.
3. **Observable multivalued fragment:** collection, first-visible behavior, explicit stream materialization, and other genuinely HE-visible behaviors. This is where extra cost may be inherent.

The conceptual advance here is important: one should not ask whether “HE is slow.” One should ask which semantic fragments permit native lowering and which force richer runtime machinery.

5.3. A strong possible result

A valuable theoretical result would be to prove:

- a **positive theorem:** certain HE fragments admit semantics-preserving lowering to native Prolog execution up to guarded fallback, and
- a **separation theorem:** some HE-observable fragments necessarily require extra runtime machinery compared with plain deterministic logic execution.

This is much more meaningful than generic optimism or pessimism about HE performance.

6. From raw plural producers to sinks: spaces as sources and sinks

A recurring systems problem is the gap between plural producers and plural consumers. In many naive designs, a source like:

```
(get-atoms &src)
```

produces many results, but a consumer like:

```
(add-atoms &dst ...)
```

accepts only a reified tuple. The resulting bridge:

```
(add-atoms &dst (collapse (get-atoms &src)))
```

creates exactly the wrong materialization boundary.

The durable concept is a **source/sink protocol**. A source should not have to become a concrete tuple merely because the consumer is plural. This applies not only to atomspace transfer but later to answer streams, fold sinks, and process-event pipelines.

The clean rule is:

Do not force a plural producer through an aggregate representation unless the surface explicitly requests materialization.

This same idea will recur in process-calculus stepping, stream folds, and tabling.

7. Proof-guided optimization instead of ad hoc cleverness

The highest-leverage optimization strategy is not “invent optimizer folklore.” It is:

1. define a small trusted artifact, either a proof object or a validator,
2. treat optimization opportunities as local rewrite or translation-validation questions,
3. and use a search or equality substrate rather than embedding every optimization as hand-written special-case lore.

7.1. Three useful layers

A coherent stack has:

- a **born-canonical term layer** with maximal sharing and stable keys,
- a **proof- or validator-backed local rewrite layer**,
- and a **query/tabling/search layer** for incremental reasoning and bounded demand.

This is exactly the spirit behind proof-derived optimizations, translation validation, e-graphs, and Datalog/tabling systems.

7.2. Practical moral

The trusted object should be small. It might be:

- a proof object,
- a local validator,
- or a translation-check relation.

The point is the same: trust the proof or validator, not unstructured optimizer bravado.

8. Concurrency and distributed runtime: the hybrid answer

A serious CeTTa runtime should not force one concurrency model across all workloads. The strongest architecture is hybrid.

8.1. Single-machine default

On one machine:

- use a fixed-size **work-stealing pool** for CPU-bound symbolic work,
- use **structured query scopes** for cancellation and failure propagation,
- use **bounded channels** for answer streams and pipeline edges,
- reserve actor-style isolation for coarse services such as networking, adapters, and foreign boundaries.

8.2. Distributed default

For distributed scale-out:

- keep strong consistency for metadata and control-plane state,
- use event propagation or append-only exchange for the data plane,
- use CRDT-style replication only where the merge law is mathematically valid,
- never pretend consensus and CRDTs solve the same problem.

8.3. TableManager, AnswerBank, and streams

A future-proof architecture treats:

- `TableStore` as a real **TableManager** with active-call states,
- `AnswerBank` as an **answer publication substrate**,
- and hyperpose-like branch engines as **budgeted parallel producers/consumers** over bounded answer streams.

9. RhoCalc: the right architecture for the core calculus

9.1. The kernel first

The best architecture for `-lang rhocalc` is a calculus kernel with projections, not a CLI implementation and not a host library implementation.

The kernel should own:

- rho-native terms,
- validation,
- normalization,
- one-step reaction collection,
- and a raw successor-set result type.

A good shape is:

```
RhoTerm -> RhoStepSet
```

not:

```
RhoTerm -> superpose(...) -> CLI undoing that packaging again
```

9.2. A crucial rule

Object-level parallelism, meta-level branching, and host-level nondeterminism must stay distinct.

In other words:

- `rho:par` is object-level parallelism,
- `RhoStepSet` is the meta-level branching object,
- host `superpose` or similar nondeterminism is only an optional projection.

This rule is one of the main protection mechanisms against poisoning the architecture.

9.3. Recommended layering

1. **Kernel:** `RhoTerm`, `RhoProfile`, validator, normalizer, `RhoStepSet`.
2. **CLI:** `-lang rhocalc` as the first public surface, exposing one-step semantics by default.
3. **Translation:** `.rho <-> .mrho` with fail-closed accepted subsets.
4. **Host adapter:** `lib/rho` as a thin adapter over the kernel.
5. **Richer surface later:** `lib/rholang` or pattern libraries as a separate layer.

9.4. Validation discipline

The key anti-bug rule is:

No accepted-but-misbehaving forms. Either support a form correctly or reject it explicitly.

This is especially important around dependent joins, quoted pattern positions, and accepted concrete syntax.

9.5. Parallel-ready collector shape

For future serious rho work, the collector should already be shaped as:

- local reaction buffers,
- extensional keys,
- explicit merge/dedup,
- no hidden dependence on host packaging,
- and clear ownership of successor batches.

Even before actual parallel execution, that ownership boundary is worth getting right.

10. Self-hosting, DTT, and staged implementation

A naive dream of “write the whole thing in itself” is not the right next move. The better pattern is staged self-hosting.

10.1. Three layers

- **Native kernel:** hot runtime paths, canonical term representation, matcher kernels, storage engines, FFI boundaries.
- **CeTTa-K / CeTTa-IL:** a restricted lowerable kernel language with explicit effects and search boundaries.
- **Full reflective CeTTa:** translators, elaborators, diagnostics, proofs, richer language services.

This is the right way to think about self-hosting: front-end and middle-end first, not all hot loops immediately.

10.2. Why DTT matters first

If dependent type theory is a bigger roadmap item than self-hosting, the right move is to use DTT where it clarifies boundaries immediately:

- typed IR and elaboration,
- rewrite certificates,
- proof-directed diagnostics,
- effect distinctions,
- and typed foreign boundaries.

The first big application is not necessarily a full self-hosting compiler. It may instead be the typed bridge and elaboration infrastructure that later makes self-hosting plausible.

11. Testing, benchmarks, and conformance

The correct separation is:

tests are for correctness and capability; benchmarks are for performance.

11.1. Tests

Use explicit categories:

- **exact:** stable golden output
- **pattern:** structure matters, incidental formatting does not
- **property:** invariant-based checks
- **probe:** non-gating diagnostics
- **xfail:** known-broken or intentionally unsupported cases

Build once per **build-key**, not once per individual test:

(build mode, flags, compiler, runtime features)

Then run test subsets over that build artifact.

11.2. Benchmarks

Use tiers:

- **bench-smoke**: cheap catastrophe detector
- **bench-compare**: meaningful regression comparison with baselines
- **bench-capacity**: large or long-running workloads

Do not treat benchmark outputs like golden files. Store performance histories keyed by:

```
commit + machine + build-key + benchmark name
```

11.3. One recurring lesson

A benchmark or test lane should be kept only if its failure tells you something actionable. Bureaucratic harnesses are almost as dangerous as missing harnesses.

12. Commercial-grade versus research-grade

“Commercial-grade” is not a mystical badge and not just “software written by expensive professionals.” It is software that a team is prepared to stand behind under real user load, real failures, real upgrades, and real support obligations.

That usually requires more than a strong research core:

- release and upgrade policy,
- build/install portability,
- observability,
- support and ownership,
- security/update path,
- and a commitment to smoothing the long tail of runtime behavior.

A mature substrate like SWI-Prolog helps because it contributes a strong floor: portability, tooling, debugger/profiler infrastructure, release cadence, and a more predictable runtime base. But a language layered over it does not automatically inherit the whole commercial-grade envelope. The hosted language still has to earn its own semantics, ergonomics, compatibility story, and support obligations.

13. Anti-patterns to avoid

The following mistakes are worth naming explicitly, because they recur.

1. **Host-first kernel design**: letting the adapter define the semantic core.
2. **Runner disguised as language**: repeated execution or scheduler policy hidden behind ordinary language entrypoints.
3. **Accepted-but-wrong forms**: parsing or lowering syntax that is not truly supported.
4. **Implicit profile drift**: same syntax meaning different things under different implementations without explicit profile declarations.

5. **Optimization folklore:** trusting hand-written special cases without proofs, validators, or clear semantic contracts.
6. **Build-path mythology:** requiring personal checkout layouts or undocumented external machinery for the default user path.

14. A compact near-term roadmap

A coherent near-term roadmap looks like this.

14.1. For MeTTa/PeTTa/HE work

1. keep a strict HE-compatible core and explicit compatibility/extension rings,
2. finish deterministic compiled-call and batched mutation substrates,
3. keep visible multivalue semantics explicit,
4. treat translation-generated witnesses as semantic evidence, not only as examples.

14.2. For RhoCalc

1. freeze a pure core kernel with a raw successor-set API,
2. expose the one-step relation through CLI and host adapters without semantic divergence,
3. make accepted `.rho` subsets fail closed,
4. keep richer surfaces in `lib/rholang` or rho-pattern libraries, not in the kernel.

14.3. For CeTTa as shared substrate

1. stabilize build-key-based testing,
2. stabilize bounded streams, budgets, and runtime counters,
3. treat parallelism as a runtime substrate layered under, not as a reason to muddy semantic cores.

15. Questions that should anchor future chats

Any new design or implementation discussion should answer these questions explicitly.

1. What is the semantic object at the center of this feature?
2. Which layer owns it: kernel, syntax, profile, runtime, or runner?
3. What are the explicit acceptance/rejection boundaries?
4. What invariant test would catch this class of regression permanently?
5. Is the proposed optimization justified by a semantic classification, a proof, or a validator?
6. Is this change general substrate work, or a family-specific fix masquerading as infrastructure?
7. Will this make later concurrency/distributed work cleaner, or merely postpone the cleanup?

16. Conclusion

The main conceptual lesson is simple but demanding: keep semantic objects small, explicit, and independent of their projections. A language family like CeTTa/MeTTa/PeTTa/RhoCalc can evolve coherently only if its core relations, profiles, translations, and runtime policies are disentangled before implementation pressure makes their boundaries blurry. Once those boundaries are clean, performance work becomes more principled, standardization becomes less frightening, testing becomes more meaningful, and productization stops looking like a vague social demand and starts looking like a manageable engineering program.

Suggested working maxim: *Define the raw object first. Everything else is an adapter, proof obligation, or policy layer.*